

VESTIGIA

Project Architecture & Structure Document

AI-powered interior construction progress monitoring platform for IEVO

Repository: snag-vision-web | Branch: feature/ui-updates
Generated: 2026-08-13

1. System Overview

VESTIGIA is a full-stack web application for tracking interior construction/fit-out progress. Field teams capture photos/panoramas and 360° walkthroughs of a site; a FastAPI backend stores project/floor/unit/room hierarchy and runs AI (Google Gemini) analysis on captured media to detect construction activities and progress against an Excel-driven activity mapping. A React single-page application provides dashboards, floor plan & panorama viewers, upload flows, and an issue-tracking (snagging) module.

High-Level Architecture

Layer	Technology	Responsibility
Frontend	React 18 + Vite + Tailwind CSS + React Router	SPA UI: dashboards, floor/panorama viewers, uploads, issues
Backend API	FastAPI (Python), Uvicorn	REST API, auth, business logic, background job worker
Database	SQLAlchemy ORM (Postgres/SQL Server via psycopg2)	Relational persistence of projects, media, analyses, issues
AI / Analysis	Google Gemini (google-genai), ChromaDB	Vision-based activity/progress detection from media
Storage	Google Cloud Storage (with local /uploads fallback)	Media file storage (images, panoramas, Excel sheets)
Containerization	Docker + docker-compose	Local/dev orchestration of frontend + backend

2. Repository Structure

Top-level layout

```
snag-vision-web/  
  ■■■ backend/      # FastAPI application  
  ■■■ frontend/     # React (Vite) application  
  ■■■ docker-compose.yml  
  ■■■ README.md
```

2.1 Backend (backend/)

```
main.py          # FastAPI app entrypoint, router registration, startup hooks  
config.py        # Settings (env-driven, pydantic-settings)  
seed.py          # Demo data seeding on startup  
models/  
  ■■■ database.py   # SQLAlchemy ORM models (all tables)  
  ■■■ __init__.py   # init_db(), SessionLocal  
schemas/models.py # Pydantic request/response schemas  
routers/  
  # API route modules (see section 3)  
services/  
  # Business logic / integrations (see section 3)  
uploads/  
  # Local media storage fallback (served at /uploads)  
Dockerfile, requirements.txt
```

2.2 Frontend (frontend/src/)

```
main.jsx, App.jsx # App bootstrap & route table  
pages/            # Route-level views (see section 4)  
components/  
  # Reusable UI: Sidebar, UI kit, WalkthroughBar,  
  issues/ (Drawer, IssueListPanel, IssueDetailsPanel,  
  IssueFormPanel, IssueBits), markers/MarkerLayer  
hooks/  
  # useAuth, useProject, SiteContext, useIssues  
utils/  
  # api.js (Axios client), standardActivities.js
```

3. Backend Modules

3.1 API Routers

Router	Purpose
auth.py	User registration/login, JWT-based authentication (python-jose, passlib/bcrypt)
projects.py	CRUD for projects, floors, units, rooms
uploads.py	Media upload endpoints (images/panoramas), storage to GCS or local disk
analysis.py	Triggers/returns AI vision analysis results for uploaded media
analysis_jobs.py	Async analysis job creation and status polling
mobile.py	Endpoints tailored for mobile capture clients
site.py	Site/floor-plan and hotspot related endpoints
walkthroughs.py	Capture-session ('walkthrough') entity behind the unified media pipeline
issues.py	Two routers: generic marker layer (markers_router) + issue tracking/snagging (router)

3.2 Services (business logic)

Service	Purpose
gemini_service.py	Google Gemini vision calls — detects activities/progress in media
gcs_service.py	Google Cloud Storage upload/retrieval client
job_worker.py	In-process async polling worker that processes queued analysis jobs (single-instance-safe)
mapping_service.py	Maps detected AI activities to standard construction activity catalog
activity_catalog.py	Canonical list/definitions of trackable construction activities
excel_service.py	Parses/generates Excel activity-mapping sheets (openpyxl)
progress_service.py	Aggregates unit/room progress from activity mappings
walkthrough_service.py	Manages walkthrough/capture-session lifecycle and media association

3.3 Data Model (SQLAlchemy — backend/models/database.py)

Core entities and relationships, grouped by domain area:

- **Identity:** User (role-based: admin/PM/field, etc.)
- **Project hierarchy:** Project → Floor → Unit → Room
- **Media & capture:** MediaUpload, Spot, CaptureSession, Hotspot, HotspotCapture, FloorPlan
- **AI analysis:** AIAnalysis, AnalysisJob (status-tracked async jobs), Milestone
- **Activity tracking:** Activity, ActivityMapping, StandardComponentMapping, UnmappedComponent, UnitActivityProgress, ActivityExcelFile
- **Issue tracking (snagging):** Marker, Issue, IssueComment

Enums used across the model: UserRole, UploadStatus, AnalysisJobStatus, WalkthroughStatus, IssueStatus, IssuePriority.

3.4 Backend Request Flow

1. Client authenticates via **auth.py** (JWT issued).
2. Field user uploads media through **uploads.py** / **mobile.py** → stored on GCS or local */uploads* →

MediaUpload/Spot/CaptureSession rows created.

3. An AnalysisJob is queued (**analysis_jobs.py**) and picked up asynchronously by **job_worker.py**, which calls **gemini_service.py** to analyze the image/panorama.

4. Detected activities are reconciled against the activity catalog via **mapping_service.py** and **excel_service.py**, updating UnitActivityProgress.

5. Results are exposed through **analysis.py** / **projects.py** / **site.py** for the frontend dashboards, floor plan, and panorama viewers.

6. Field issues (snags) are logged as Markers/Issues via **issues.py**, tied to a spot/room.

4. Frontend Modules

4.1 Routing (App.jsx)

Route	Page Component	Access
/login	Login.jsx	Public
/register	Register.jsx	Public
/dashboard	Dashboard.jsx (DashboardNew.jsx in progress)	Protected
/floors	FloorView.jsx (FloorViewNew.jsx / FloorMap.jsx variants)	Protected
/upload	Upload.jsx	Protected
/analysis	Analysis.jsx	Protected
/issues	Issues.jsx	Protected
/panorama	PanoramaViewer.jsx	Protected
/projects	Projects.jsx	Protected
/site-setup	SiteSetup.jsx	Protected
/site-capture	SiteCapture.jsx	Protected
* (fallback)	Redirect → /dashboard	-

Protected routes are wrapped in **ProtectedRoute** (checks `useAuth`), then **ProjectProvider** and **SiteProvider** context providers, then the **Layout** (Sidebar + main content area).

4.2 State & Context (hooks/)

Hook / Context	Purpose
<code>useAuth.jsx</code>	<code>AuthProvider</code> — login/register/session state, JWT storage, current user
<code>useProject.jsx</code>	<code>ProjectProvider</code> — selected project, floors/units/rooms context
<code>SiteContext.jsx</code>	<code>SiteProvider</code> — active site/capture-session state for setup & capture flows
<code>useIssues.js</code>	Data hook for fetching/mutating issues & markers

4.3 Components

- **Sidebar.jsx** — primary navigation
- **UI.jsx** — shared UI kit (buttons, cards, inputs, etc.)
- **WalkthroughBar.jsx** — walkthrough/capture-session controls
- **markers/MarkerLayer.jsx** — renders interactive markers over floor plans/panoramas
- **issues/** — Drawer, `IssueListPanel`, `IssueDetailsPanel`, `IssueFormPanel`, `IssueBits` (issue tracking UI, used by `Issues.jsx`)

4.4 Utilities & Key Libraries

File / Library	Purpose
<code>utils/api.js</code>	Centralized Axios HTTP client (base URL, interceptors)
<code>utils/standardActivities.js</code>	Frontend copy of standard activity list for UI mapping
<code>three</code>	3D/panorama rendering (used by <code>PanoramaViewer</code>)

File / Library	Purpose
recharts	Dashboard charts/progress visualization
xlsx / pdfjs-dist	Client-side Excel and PDF handling (floor plans, activity sheets)
react-dropzone	Drag-and-drop media upload UI
react-hot-toast	Toast notifications
tailwindcss	Utility-first styling

5. Deployment & Configuration

- **docker-compose.yml** at repo root orchestrates the frontend and backend containers together for local/dev use.
- **backend/Dockerfile** builds the FastAPI service (Uvicorn); **frontend/Dockerfile** builds/serves the Vite app.
- Backend configuration is environment-driven via **backend/config.py** (pydantic-settings) reading **backend/.env** (see **.env.example**) — DB connection, GCS credentials, Gemini API key, CORS origins.
- Database access supports both Postgres (psycopg2) and SQL Server (pyodbc) drivers, with Alembic available for migrations.
- Media storage defaults to Google Cloud Storage, falling back to local disk served at the **/uploads** static path when GCS isn't configured.

6. Notable Architectural Points

- Async analysis is decoupled from the upload request via an **AnalysisJob** queue processed by an in-process polling worker (**job_worker.py**), started as an asyncio task on FastAPI startup — simple and effective for single-instance deployments, but not horizontally scalable as-is.
- The data model cleanly separates the physical hierarchy (Project/Floor/Unit/Room) from the capture layer (MediaUpload/CaptureSession/Hotspot) and the analysis/tracking layer (AIAnalysis/ActivityMapping/UnitActivityProgress), letting AI results be reconciled against a configurable Excel-driven activity catalog.
- Issue tracking (snagging) is modeled as a separate Marker + Issue + IssueComment subsystem, reusing the generic marker layer so markers can support both AI-detected spots and manually logged issues.
- Frontend has parallel/legacy page variants in flight (Dashboard vs DashboardNew, FloorView vs FloorViewNew vs FloorMap), consistent with the current feature/ui-updates branch being an active UI migration rather than a finished state.

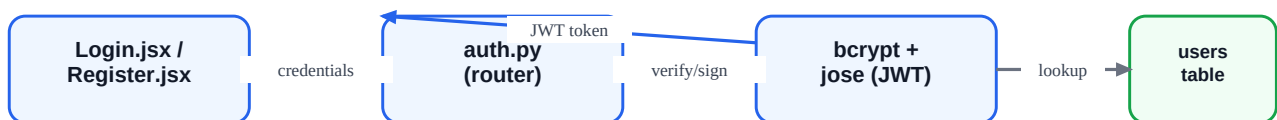
7. Data Flow Diagrams

7.1 High-Level System Data Flow



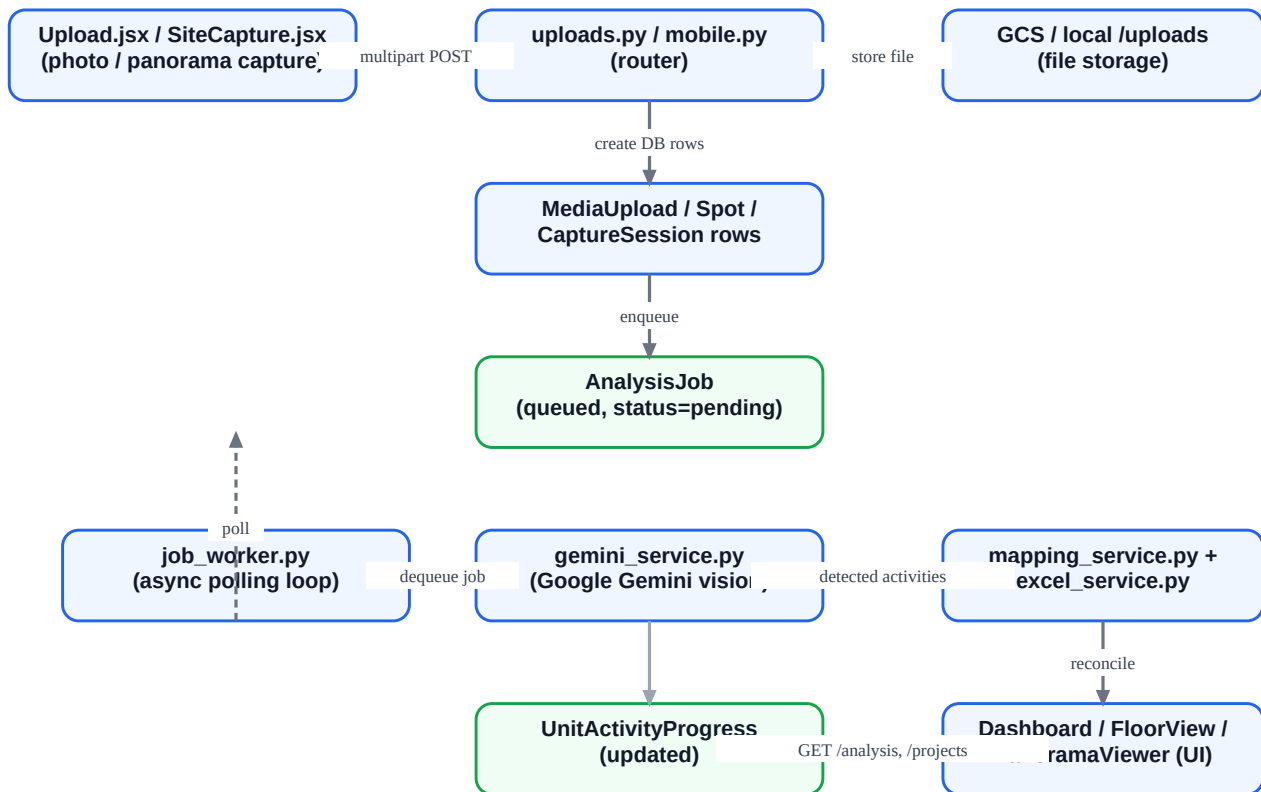
Browser se React SPA request bhejta hai FastAPI backend ko (Axios/api.js ke through). Backend SQLAlchemy se database me read/write karta hai, aur media files ko GCS (ya local /uploads fallback) me store/retrieve karta hai.

7.2 Authentication Flow



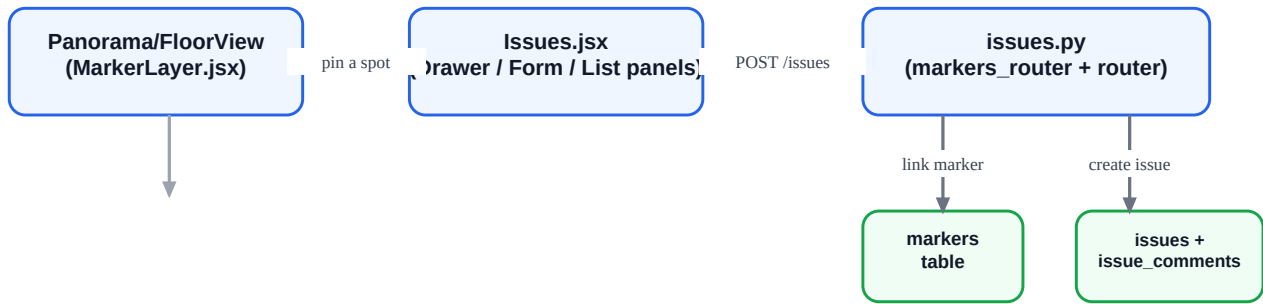
Login/Register form → auth.py verifies credentials (bcrypt) against the users table → issues a JWT (python-jose) → token useAuth.jsx (AuthProvider) me store hota hai aur har subsequent request me header me bheja jata hai; ProtectedRoute isko check karke hi page render karta hai.

7.3 Media Upload → AI Analysis → Progress Flow



- 1) User photo/panorama upload karta hai → uploads.py file ko GCS/local disk me save karke MediaUpload/Spot/CaptureSession record banata hai.
- 2) Ek AnalysisJob queue me daala jata hai.
- 3) job_worker.py background me poll karke job pick karta hai aur gemini_service.py se AI vision analysis karwata hai.
- 4) Detected activities ko mapping_service.py + excel_service.py activity catalog ke against reconcile karte hain aur UnitActivityProgress update hota hai.
- 5) Frontend (Dashboard, FloorView, PanoramaViewer) ye updated progress ko API se fetch karke dikhata hai.

7.4 Issue Tracking (Snagging) Flow



Field user floor plan ya panorama par ek spot pin karta hai (MarkerLayer.jsx) → Issues.jsx form panel se issue details (status, priority, comments) submit hoti hai → issues.py backend me Marker row aur uske sath linked Issue/IssueComment rows create/update karta hai, jo IssueListPanel/IssueDetailsPanel me wapas fetch hoke dikhaya jata hai.